SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4303
M.Madhu Rushwika(192371066)
Marks:86

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 30**

1: Online Symposium Portal

A college rv

vis developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
   <input type="text" name="name">
   <input type="email" name="email">
   <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I M.Madhu Rushwika(192371066) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M.Madhu Rushwika(192371066)

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	21.5	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	21.5	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	17.5	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	17.5	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	8.6	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	4.5	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Question 1: Online Symposium Portal

Given: a registration form (`action="/register" method="POST"`) with name and email inputs and a Submit button.

1. Missing HTML5 semantic elements

The given snippet is only a bare form. It has no document structure and none of the HTML5 semantic elements that give meaning to the page. The following are missing:

- **<!DOCTYPE html>, <html lang>, <head>, <meta charset>, <title>** - document skeleton, encoding, and page title.
- **<header>** - symposium name / logo / heading.
- **<nav>** - navigation links (Home, Schedule, Register, Contact).
- **<main>** - the primary content of the page (only one per page).
- **<section>** - a thematic block, here the registration section.
- **<fieldset> and <legend>** - group and describe related form controls.
- **<label>** - text labels tied to inputs (missing entirely).
- **<footer>** - contact details, copyright, organiser info.

Also missing: the button has no type, and the inputs have no id, requiredor placeholderattributes.

2. Modified, standards-compliant structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Symposium Registration</title>
</head>
<body>
  <header>
    <h1>National Level Technical Symposium</h1>
    <nav>
      <a href="/">Home</a> | <a href="/schedule">Schedule</a> |
      <a href="/contact">Contact</a>
    </nav>
  </header>
```

```
  <main>
    <section aria-labelledby="reg-title">
      <h2 id="reg-title">Student Registration</h2>
      <form action="/register" method="POST">
        <fieldset>
          <legend>Participant Details</legend>

          <label for="name">Full Name:</label>
          <input type="text" id="name" name="name"
                 placeholder="Enter your full name" required>

          <label for="email">Email Address:</label>
          <input type="email" id="email" name="email"
                 placeholder="you@example.com" required>

          <button type="submit">Submit</button>
        </fieldset>
      </form>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 College Symposium Committee</p>
  </footer>
</body>
</html>
```

Why it is standards-compliant: it has a DOCTYPE, a language attribute, a declared character set, a title, properly nested and closed tags, semantic containers, labelled inputs and an explicit type="submit" button. It validates in the W3C Markup Validator.

1. Interpreting the HTTP request generated on submission

Because method="POST", the browser sends the form data in the **request body** (not in the URL). Only fields with a name attribute are sent, encoded as application/x-www-form-urlencoded by default. For example, if the student enters "Ravi Kumar" and "ravi@example.com":

```
POST /register HTTP/1.1
Host:
symposium.college.edu
Content-Type: application/x-www-form-
urlencoded Content-Length: 41
User-Agent: Mozilla/5.0 ...
Accept: text/html,application/xhtml+xml
Cookie: sessionid=abc123

name=Ravi+Kumar&email=ravi%40example.com
```

Part	Meaning
Request line	**POST** = method, **/register** = resource taken from the form action, **HTTP/1.1** = protocol version.
Headers	Host (target server), Content-Type (how the body is encoded), Content-Length (body size), User-Agent, Accept, Cookie.
Blank line	Separates headers from the body.
Body	Key=value pairs joined by &. Spaces become "+" and special characters are percent-encoded (@ becomes %40).

Set-Cookie: sessionid=abc123; HttpOnly; Secure

<html> ... Registration successful! ... </html>

Status code	Meaning in this scenario
200 OK	Registration processed, confirmation page returned.
302 / 303 Redirect	Server redirects to a "Thank you" page (Post/Redirect/Get pattern, prevents duplicate submission on refresh).
400 Bad Request	Missing or invalid data.
404 Not Found	/register does not exist on the server.
500 Internal Server Error	Server-side failure (e.g., database down).

POST is used because registration **creates or changes data** on the server, it is not idempotent, and it keeps personal details out of the URL, browser history and server logs.

HTTP response cycle from server to browser

- **Step 1 - DNS and TCP:** the browser resolves the domain to an IP address and opens a TCP connection (plus a TLS handshake if HTTPS).
- **Step 2 - Request sent:** the POST request from part 3 is transmitted to the web server.
- **Step 3 - Server processing:** the web server (Apache/Nginx) passes it to the server-side program mapped to /register(PHP, Node.js, Java Servlet, etc.). The program validates the input, checks for duplicates, and stores the record in the database.
- **Step 4 - Response built:** the server creates an HTTP response with a status line, headers and a body.
- **Step 5 - Response received:** the browser reads the status code, processes headers (cookies, caching), parses the HTML body, builds the DOM, and renders the confirmation page.

HTTP/1.1 200 OK
Content-Type: text/html;
charset=UTF-8 Content-Length: 51

HTTP is **stateless**: each request/response pair is independent, so the server uses cookies or sessions to remember the user between requests.

2. Improvements for usability and accessibility

Usability

- Use required, pattern, minlengthfor instant client-side validation, and repeat validation on the server.
- Add placeholderhints, autocomplete="name"and autocomplete="email"to speed up input.
- Add more useful fields: phone, college, department, event selection (<select>, radio buttons).
- Show clear inline error messages and a success message after submission.
- Use a descriptive button label such as "Register Now" and a responsive layout for mobile users (viewport meta tag, flexible CSS).

Accessibility

- Associate every input with a <label for="id">so screen readers announce it and clicking the label focuses the field.
- Group controls with <fieldset>/<legend>.
- Use ARIA where needed (aria-required, aria-describedbyfor hints and role="alert"for errors).
- Ensure sufficient colour contrast (WCAG 4.5:1), a visible focus outline, and full keyboard operability (logical tab order).
- Set lang="en"on the html element, and provide alttext for any images.

Question 2: Cross-Browser Compatibility Issue

Given: an HTML page whose `<h1>` and `<p>` tags are not closed, and which lacks a DOCTYPE, semantic elements, and a charset declaration.

1. Improper nesting and missing closing tags

Problem	Explanation
<h1> not closed	The line Welcomehas no .
** not closed**	The line About ushas no .
Improper nesting	A is placed *inside* an unclosed <h1>. A paragraph (flow content) cannot be a child of a heading (phrasing content only). The then tries to close elements that are still open.
No DOCTYPE	Without <!DOCTYPE html>browsers switch to *quirks mode*.
Missing meta charset / lang	No <meta charset>and no langattribute.

Note: HTML5 allows the end tag of to be omitted in some cases, but **<h1> has no such permission**, and relying on omitted tags is poor practice, especially with XHTML/XML parsers, where it is a fatal well-formedness error.

2. How different browsers may interpret the structure

Browsers do not reject invalid HTML. They apply **error-recovery (tag-soup) rules** to repair it, and the repair is not always identical:

- **Quirks mode:** with no DOCTYPE, browsers emulate old behaviour. Box model, default font sizes, table inheritance and line-height can differ between Chrome, Firefox, Safari and (older) IE, so the layout looks different.
- **Auto-closing:** when the parser meets inside <h1>, most modern HTML5 parsers implicitly close the <h1> before the . The resulting DOM is: h1 "Welcome"followed by p "About us". Older or non-conforming parsers may keep the nested inside the heading, so it inherits the heading font size and bold style.
- **Different DOM trees** lead to different CSS selector matches (e.g., h1 p matches in one browser but not another), different margins (margin collapsing), and JavaScript DOM traversal returning different results.

- **Character encoding:** without a charset, the browser guesses (sniffing), so special characters may display incorrectly.
- **Strict parsers (XHTML/XML):** an XML parser will refuse to render and show a parsing error because the document is not well-formed.

Conclusion: the inconsistent layout occurs because each browser silently "fixes" the broken markup in its own way. Valid, well-formed HTML gives every browser the same DOM.

3. Missing semantic elements

- **<header>** - for the site heading / logo ("Welcome").
- **<nav>** - main navigation menu.
- **<main>** - main page content.
- **<section> / <article>** - the "About us" content as its own thematic section.
- **<footer>** - contact and copyright information.
- Using a generic tells browsers, search engines and screen readers nothing about the purpose of the content.

4.Corrected W3C-compliant HTML structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company</title>
</head>
<body>
  <header>
    <h1>Welcome</h1>
    <nav>
      <ul>
        <li><a href="#about">About Us</a></li>
        <li><a href="#contact">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section id="about">
      <h2>About Us</h2>
      <p>We are a company committed to quality and innovation.</p>
    </section>
  </main>
```

```
    <footer>
      <p id="contact">&copy; 2026 Company. All rights reserved.</p>
    </footer>
  </body>
  </html>
```

What was fixed: DOCTYPE added, lang and charset declared, every tag closed, correct nesting (heading and paragraph are siblings, not parent and child), semantic elements replace the generic div, and a proper heading hierarchy (h1 then h2) is used.

5.Techniques to ensure cross-browser compatibility

- **Validate markup and CSS** using the W3C Markup Validator and CSS Validator; always include the DOCTYPE to trigger *standards mode*.
- **Use a CSS reset or normalize.css** to remove default style differences between browsers.
- **Use semantic HTML5** and, if legacy support is needed, an HTML5 shim (html5shiv) for old IE.
- **Responsive design:** viewport meta tag, relative units (%, rem, em), Flexbox/Grid, and media queries.
- **Vendor prefixes and fallbacks:** use Autoprefixer for -webkit-, -moz-prefixes; provide fallback values and use @supports for feature detection.
- **Feature detection instead of browser sniffing** (e.g., Modernizr); progressive enhancement / graceful degradation.
- **Check support** on caniuse.com and MDN before using a new feature; use Babel/polyfills for modern JavaScript.

- **Test** on multiple browsers and devices (Chrome, Firefox, Safari, Edge, mobile) using BrowserStack, LambdaTest or built-in DevTools device emulation.
- **Declare encoding** with <meta charset="UTF-8">and serve the correct Content-Type header.
- **Use standard, well-supported technologies**; avoid deprecated tags such as <font> and <cente

Question 3: Form Submission Failure

Given: a login form with `action="/submit" method="GET"`, username and password inputs, and `<button type="button">Login</button>`.

1. Issues in the form configuration

#	Issue	Why it is a problem
1	**<button type="button">**	A "button"-type button has **no default behaviour**. It does not submit the form. This is the main cause of the failure.
2	**method="GET"** for login	GET appends credentials to the URL, which exposes the password in browser history, server logs, the Referer header and bookmarks. Wrong method for sensitive data.
3	**No labels**	Inputs have no <label>, so this is poor for accessibility and usability.
4	**No validation attributes**	No required, so empty data can be submitted.
5	**No autocomplete hints**	Missing autocomplete="username"/ "current-password", so password managers work poorly.
6	**Possibly HTTP, no CSRF token**	Nothing shows that HTTPS or CSRF protection is in place.

2. Why the HTTP request is not triggered

An HTML form is submitted only when a **submit control** is activated: <button type="submit">, <input type="submit">, a button with no type specified (default is submit), or when form.submit() is called from JavaScript.

- Here the button is type="button", which is a plain push-button with no associated action.
- Clicking it fires only a *click* event; there is no *submit* event, so the browser never builds the request.
- Since no JavaScript handler (onclickcalling this.form.submit()or fetch()) is attached either, nothing happens.

- As a result, no URL is composed, no TCP/HTTP request is sent, and the server never receives username or password. The failure is entirely on the client side.

3. Role of the GET method in this scenario

If the button were fixed to submit, the GET method would work like this: the browser would append the data to the URL as a **query string** and send a request with no body:

```
GET
/submit?username=admin&password=MyPass123 HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0 ...
```

Characteristic of GET	Effect on a login form
Data in the URL (visible)	Password shown in address bar, history and bookmarks **- security risk**.
Logged everywhere	Web-server, proxy and analytics logs record full URL including password.
Cached and bookmarkable	Meant for safe, read-only retrieval (search, filters), not for authentication.
Idempotent / safe	GET should not change server state; login creates a session, so it should be POST.
Length limits	URL length is limited (about 2000 characters in practice).

Characteristic of GET	Effect on a login form
Leaks via Referer	The URL may be sent to third-party sites through the Referer header.

Conclusion: GET is appropriate for search forms and filters but is the wrong choice for credentials. Login forms should use **POST** over **HTTPS**.

4. Corrected implementation

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>Login</title>
  </head>
  <body>
    <main>
      <h1>User Login</h1>
      <form action="/submit" method="POST">
        <fieldset>
          <legend>Sign in to your account</legend>

          <label for="username">Username:</label>
          <input type="text" id="username" name="username" autocomplete="username"
                 required>

          <label for="password">Password:</label>
          <input type="password" id="password"
                 name="password" autocomplete="current-
                 password" minlength="8" required>

          <button type="submit">Login</button>
        </fieldset>
      </form>
    </main>
  </body>
  </html>
```

Changes made: (1) type="button"changed to type="submit", which triggers the request; (2) GET changed to POST so credentials travel in the request body; (3) labels, required, minlengthand autocompleteadded; (4) fieldset/legend and a complete valid HTML5 skeleton added.

The request now generated is:

```
POST /submit HTTP/1.1
Host:
www.example.com
Content-Type: application/x-www-form-
urlencoded Content-Length: 34

username=admin&password=MyPass123
```

5. Improvements for secure data transmission

- **Use HTTPS (TLS)** so the request is encrypted in transit; redirect all HTTP traffic to HTTPS and enable HSTS (Strict-Transport-Securityheader). Set the form action to an https URL.

- **Use POST, never GET,** for credentials so they stay out of URLs and logs.
- **Hash passwords on the server** with bcrypt, scrypt or Argon2 with a salt; never store plain text.
- **CSRF protection:** include an anti-CSRF token in a hidden field and use SameSite cookies.
- **Secure cookies:** set HttpOnly, Secureand SameSite=Strict/Laxon the session cookie.
- **Validate and sanitise input** on both client and server to prevent SQL injection and XSS; use prepared statements.
- **Brute-force defence:** rate limiting, account lockout, CAPTCHA and multi-factor authentication (MFA/OTP).
- **Security headers:** Content-Security-Policy, X-Frame-Options, X-Content-Type-Options; add Cache-Control: no-storefor sensitive pages.
- **Generic error messages** such as "Invalid username or password", so attackers cannot tell which one was wrong.

Avoid exposing credentials in logs, URLs or JavaScript; use type="password"to mask inpu